

Realisation

Student Name: David Hadzhiivanov

Student Number: 5670330

Table of Contents

Realisation	1
Table of Contents.....	1
1. Introduction	1
2. Development Workflow & Version Control	1
3. Source Code	1
3.1 File Structure Overview	1
3.2 Core Features	2
3.3 Key Routes and Behaviour	3
4. Frontend Implementation.....	3
5. Hosting and Deployment	6
6. Conclusion.....	7

1. Introduction

This document explains how the Secure Notepad Web Application was realised from code to running system. It describes the structure of the application, the main technical components, how the core features are implemented, and how the app is deployed and served securely.

2. Development Workflow & Version Control

The project was developed using **Git** for version control. The repository contains a full commit history. Using **Git** allowed structured commits for feature work and clean rollbacks when refactoring was necessary.

Development workflow notes:

- Local development used an application factory pattern to run the Flask app during development.

- Dependencies and environment were handled via a virtual environment (venv) during development.
- Code was iteratively updated and tested locally before deployment to the Raspberry Pi host.

3. Source Code

3.1 File Structure Overview

This structure keeps presentation (templates), logic (blueprints), persistence (db), and utilities (encryption) separated and easy to follow.

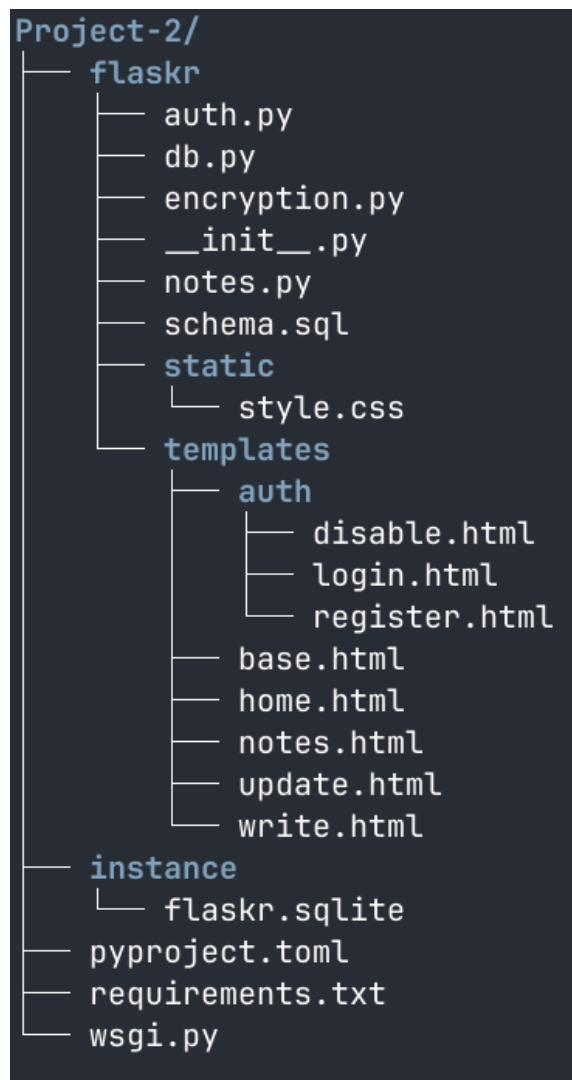


Figure 1. File structure

3.2 Core Features

- **Authentication:** Users register and log in; passwords are hashed before storage using Werkzeug security.
- **Note management:** Notes are created, listed, edited and deleted via routes in notes.py. Each note is associated with its author in the database.
- **Encryption:** Notes are encrypted before storage using the substitution cipher module (flaskr/encryption.py). The user provides or generates a key; the server stores only ciphertext.
- **Database:** SQLite is used as the lightweight persistence layer with a user table and a post table (posts used as notes). The schema is in flaskr/schema.sql. DB access uses a get_db() helper that reuses a connection per request.

```
flaskr > schema.sql
1  DROP TABLE IF EXISTS user;
2  DROP TABLE IF EXISTS post;
3
4  CREATE TABLE user (
5      id INTEGER PRIMARY KEY AUTOINCREMENT,
6      username TEXT UNIQUE NOT NULL,
7      password TEXT NOT NULL
8  );
9
10 CREATE TABLE post (
11     id INTEGER PRIMARY KEY AUTOINCREMENT,
12     author_id INTEGER NOT NULL,
13     created TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
14     title TEXT NOT NULL,
15     body TEXT NOT NULL,
16     FOREIGN KEY (author_id) REFERENCES user (id)
17 );
```

Figure 2. schema.sql snippet

3.3 Key Routes and Behaviour

Core endpoints (high-level):

- **/auth/register** - register new user (POST username+password; store hashed password).
- **/auth/login** - authenticate user (POST username+password; set session).
- **/auth/logout** - clear session.

- **/posts** - list all notes for the logged-in user.
- **/write** (or equivalent) - create a new note: plaintext + user key → server runs encryption utility → saves ciphertext to DB.
- **/<int:id>/update** - view and edit a note. The route supports decryption by accepting a key from the user and returning plaintext if the key is correct; it also supports saving edited content (which is re-encrypted before storage).
- **/<int:id>/delete** - delete a note.
- **/delete_account** - delete_account page.

Routes are secured by a login_required decorator, which checks the session and prevents unauthorized access.

4. Frontend Implementation

- **Templates:** Jinja2 templates are used for rendering pages: login, register, home, posts list, write, update.
- **UX:** Minimal, single-column layout focused on clarity. Forms are used for authentication.
- **Data flow:** Forms POST to Flask routes; server returns rendered templates. Decryption flow is performed when a user supplies a key in the update/view template - the template displays decrypted content if the server decrypt succeeds.

Home page:

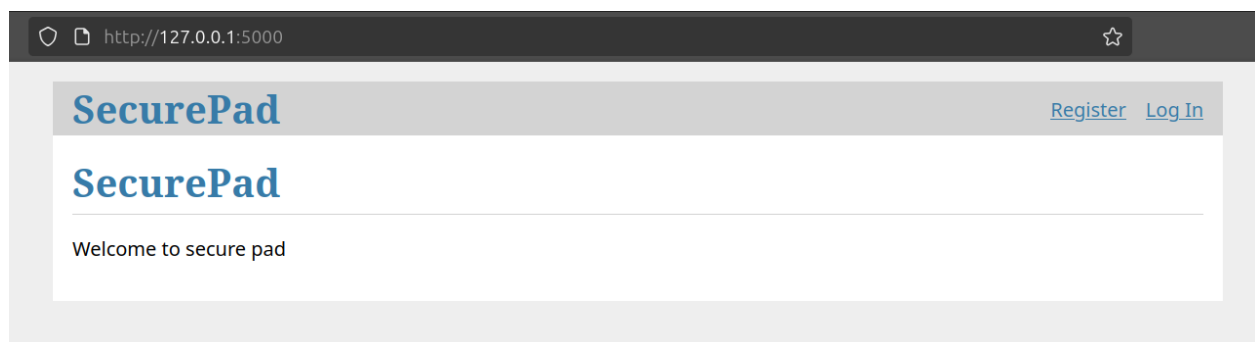


Figure 3. Home page layout final design

Registration / Login page:

The screenshot shows a web browser window with the address bar displaying `http://127.0.0.1:5000/auth/register`. The page has a dark header bar with the site name "SecurePad" on the left and two links, "Register" and "Log In", on the right. Below the header, the main content area has a light gray background. It features a large blue heading "Register". Underneath, there are two form fields: "Username" and "Password", each with a text input box. A "Register" button is positioned below the password field.

Figure 4. Registration page layout final design

The screenshot shows a web browser window with the address bar displaying `http://127.0.0.1:5000/auth/login`. The page has a dark header bar with the site name "SecurePad" on the left and two links, "Register" and "Log In", on the right. Below the header, the main content area has a light gray background. It features a large blue heading "Log In". Underneath, there are two form fields: "Username" and "Password", each with a text input box. A "Log In" button is positioned below the password field.

Figure 5. Login page layout final design

Notes page:



Figure 6. Notes page layout final design

Decrypt notes and edit:

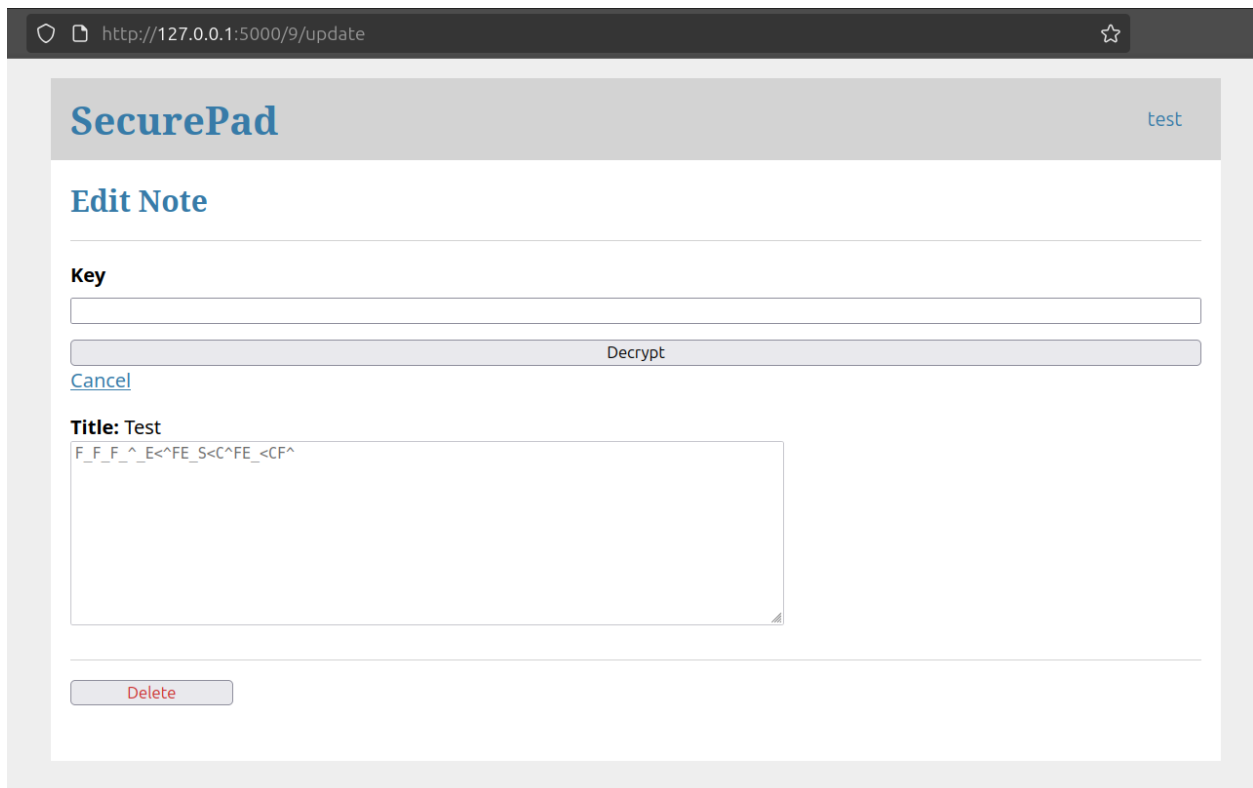


Figure 7. Decrypt note page before editing layout final design

The screenshot shows a web browser window with the address bar displaying `http://127.0.0.1:5000/11/update`. The page has a header with the logo "SecurePad" on the left and the text "test" on the right. Below the header, the main content area is titled "Edit Note". It contains two form fields: "Title" with the value "Test" and "Message" with the value "Test". At the bottom of the form, there are three buttons: "Save", "Cancel", and "Delete". The "Save" and "Cancel" buttons are grouped together, and the "Delete" button is positioned below them.

Figure 8. Edit note page layout final design

Delete account page:

The screenshot shows a web browser window with the address bar displaying `http://127.0.0.1:5000/auth/delete_account`. The page has a header with the logo "SecurePad" on the left and the text "Demo2" on the right. Below the header, the main content area is titled "Delete Account". It contains a warning message: "This will permanently remove your account and related data. You will be logged out." Below the message, there is a form field labeled "Confirm your password". At the bottom of the form, there are two buttons: "Delete Account" and "Cancel".

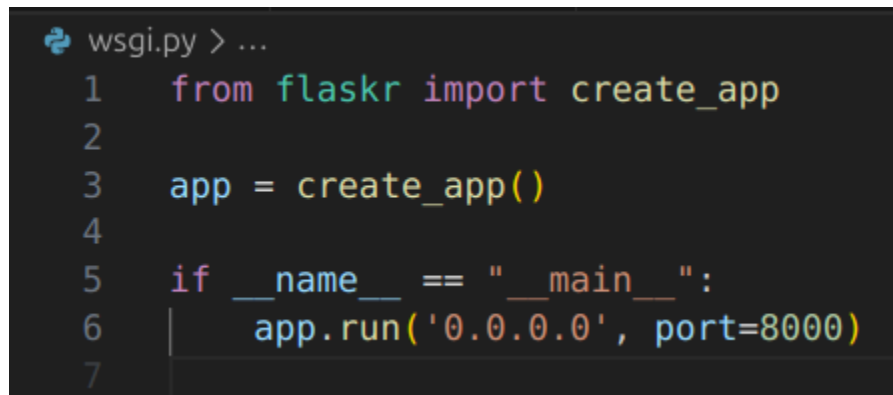
Figure 9. Delete account page layout final design

5. Hosting and Deployment

The application was deployed on a **Raspberry Pi** with **NGINX** as reverse proxy and **Gunicorn** (or equivalent WSGI server) to serve the Flask app. **HTTPS** was enabled via **Certbot** for automated certificate issuance.

Deployment essentials:

- **WSGI** entrypoint is `wsgi.py` which uses `create_app()` to run the app at `0.0.0.0` on a chosen port.



```
wsgi.py > ...
1  from flaskr import create_app
2
3  app = create_app()
4
5  if __name__ == "__main__":
6      app.run('0.0.0.0', port=8000)
7
```

- **NGINX** configuration forwards incoming requests to the local Gunicorn socket or port; Certbot configured SSL certificates and HTTPS redirect rules.
- **Firewall** and port forwarding were configured to allow external access as needed for demonstrations.
 - o The firewall used was UFW and was configured to allow access only through designated ports.
 - o Ports opened were 3 in total allowing for SSH, HTTP and HTTPS connection individually, where NGINX and Gunicorn and Certbot together manage whether a user will be routed to the HTTP or HTTPS (if available).

6. Conclusion

The realisation of the Secure Notepad Web Application resulted in a maintainable, well-structured Flask app that demonstrates applied cybersecurity concepts: secure authentication, encrypted note storage, and HTTPS deployment. The codebase organizes responsibilities cleanly across modules (auth, blog/notes, db, encryption), and the deployment shows practical skills in server configuration and certificate management. This final product is ready for validation and demonstration.